Scrabble01.java

[redacted]

Enum Level

Die Enumeratione ‚Level", wird verwendet, um die Schwierigkeitseinstellungen des Srabble-Spiels vor dem Spielbeginn einstellen zu können. Je nach Level, sind die Häufigkeiten der im Buchstaben-Säckchen jedes Spielers unterschiedlich verteilt. Außerdem variiert die Häufigkeit, mit der die Hilfsfunktion verwendet werden darf.

```
enum Level{
    EASY, NORMAL, HARD;
}
```

Klasse Bag

Die Klasse Bag, besitzt eine Instanz von Level, um die Frequenzen der Buchstaben (wie oben beschrieben) bestimmen zu können. Standartmäßig wird das Spiel im Konstruktor mit der Schwierigkeit “normal” initialisiert. Daraufhin wird entsprechend dem Level ‚normal" die Frequenz gesetzt.

```
    Bag(){
        this.level = Level.NORMAL;
        setFrequnecies();
        this.fillBag();
    }
```

Außerdem gibt es eine ArrayList bag1 und bag2, welche dann jeweils den zwei Spielenden zugewiesen wird und mit den Buchstaben befüllt werden. Dazu gibt es eine HashMap charFrequencies, welche in der Methode getFrequencies mit den Buchstaben des Alphabets und deren Häufigkeit gefüllt wird.

```
    void setFrequnecies(){
        switch(level){
            case EASY:
                this.frequencies = new int[]{10, 2, 1, 2, 15, 1, 1
                break;
            case NORMAL:
                this.frequencies = new int[]{8, 2, 2, 4, 12, 2, 3,
                break;
            case HARD:
                this.frequencies = new int[]{5, 4, 5, 3, 7, 3, 4,
                break;
        }
        getFrequencies();
    }
    void getFrequencies() {
        int count = 0;
        for (int i = 0; i < this.alphabet.length; i++) {
            this.charFrequency.put(this.alphabet[i], this.frequenc
            this.availableLetters.add(this.alphabet[i]);
            count += this.frequencies[i];
        }
    }
```

Daraufhin werden in der Methode fillBag() die beiden Bags der Reihe nach mit den Buchstaben befüllt.

Klasse Player01

Jeder Spieler besitzt einen Score, einen bag und ein Methode setBag() um den bag als eine bestimmte ArrayList zu setzen.

```
class Player01 {

    int score = 0;
    List <Character> bag = new ArrayList<>(); //Player1.bag = Bag.

    void setBag(List<Character> sourceBag) {
        this.bag = new ArrayList<>(sourceBag);
    }
```

```
}
```

Klasse Position

Die Klasse Position hat ein x und y für die jeweilige Position im Feld. Außerdem zwei boolean-Felder, um zu markieren, welcher Spieler einen Stein auf die Position gelegt hat. Zum Vergleich der Position-Objekte, gibt es eine equals-Methode, die auf den x und y Wert prüft.

```
@Override
public boolean equals(Object other) {
    if (this == other) return true;
    if (other == null || getClass() != other.getClass()) retur
    Position position = (Position) other;
    return this.x == position.x && this.y == position.y;
}
```

equals

Da ich später ein HashSet verwende, um doppelte Elemente zu vermeiden, habe ich eine Hashcode-Methode erstellt.

```
@Override
public int hashCode() {
    return Objects.hash(x, y);
}
```

compareTo

In der compareTo-Methode, werden die Position-Objekte aufsteigend entweder nach x oder y sortiert. Dies ist sinnvoll, da ein Wort entweder in einer Reihe oder in einer Spalte steht, wodurch entweder die x-Werte alle gleich sind oder die y-Werte, während sich nur jeweils die y-Werte oder x-Werte voneinander unterscheiden.

```
    @Override
public int compareTo(Position other) {
    // Vergleiche nur, wenn x gleich ist, dann nach y sortieren
```

```
        if (this.x == other.x) {
            return Integer.compare(this.y, other.y);
        }
        // Wenn y gleich ist, dann nach x sortieren
        if (this.y == other.y) {
            return Integer.compare(this.x, other.x);
        }
        // Falls weder x noch y gleich ist
        throw new IllegalArgumentException("Positionen sind nicht verg
    }
```

Klasse Word

Die Klasse Word besitzt ein Feld für die Start- und Endposition eines Wortes, eine ArrayList für alle Positionen eines Wortes, sowie einen String für das Wort selbst. Eine Methode, um Positionen zur ArrayList positions hinzufügen zu können. Hier wird nach jedem einfügen sortiert. Außerdeme eine Methode clearPosition(), um alle Positionen wieder löschen zu können.

```
    void addPosition(int x, int y){
        this.positions.add(new Position(x, y));
        Collections.sort(this.positions);
    }

    void clearPositions(){
        this.positions = new ArrayList<>();
    }
```

equals

In der equals-Methode für Word wird auf die Start- und Endposition, dem String und allen Positionen aus der Liste geprüft.

```
    @Override
    public boolean equals(Object other) {
        if (other == null) return false; // Null abwehren!
        if (other == this) return true; // Bin ich's selbst?
        if (other.getClass() != getClass()) return false; // Ander
```

```
        Word that = (Word)other; // Casting
        return Objects.equals(this.word, that.word) &&
           Objects.equals(this.start, that.start) &&
           Objects.equals(this.end, that.end) &&
           Objects.equals(this.positions, that.positions); // Was
    }
```

Es gibt außerdem wieder eine hash-Methode, welche genauso aufgabaut ist wie in der Klasse Position.

Klasse Scrabble01

Scrabble01 Konstruktor

Im Konstruktor wird das Feld gezeichnet und die zweidimensionalen Arrrays für currentBoard, updatedBoard und board initialisiert. Dabei ist das currentBoard dazu da, die im selben Zug gelegten Steine anzuzeigen, um nach Validierung den Score hochzuzählen. UpdatedBoard beinhaltet alle bereits gelegten und validierten Wörter sowie die gerade gelegten Buchstaben, um daraus alle Wörter extrahieren zu können. Wenn alle extrahierten Wörter validiert werden, bleiben die gerade gelegten Buchstaben auf dem updatedBoard und werden auf das board geschrieben. Werden die extrahierten Wörter nicht validiert, dann werden die gerade gelegten Buchstaben wieder vom updatedBoard gelöscht und eine Fehlermeldung ausgegeben. Das board existiert auch um den festen Stand des Spiels zu dokumentieren, aber hauptsächlich, um die Werte der Felder darzustellen. Es gibt “DL” = doppelter Buchstabenwert “TL” = dreifacher Buchstabenwert “DW” = doppelter Wortwert “TW” = dreifacher Wortwert

Mithilfe des boards wird der Score jedes Spielers nach seinem Zug mit der updateScore()-Methode berechnet und auf dem Spielfeld gezeichnet.

```
int updateScore(Word word){

    int startX = word.start.x;
    int startY = word.start.y;
    int endX = word.end.x;
    int endY = word.end.y;
```

```
        int mulValue = 1;
        int score = 0;

        if(startX == endX){
            for (int i = startY; i <= endY; i++) {
                score += addiereScore(this.currentBoard[startX][i], st
                mulValue += mulScore(startX, i);
            }
        } else if(startY == endY){
            for (int i = startX; i <= endX; i++) {
                score += addiereScore(this.currentBoard[i][startY], i,
                mulValue += mulScore(i, startY);
            }
        }
        if(mulValue > 1){
            mulValue--;
        }
        return mulValue * score;
    }


    int addiereScore(String currentLetter, int x, int y){
        int counter = 0;

        if(letterScores.get(currentLetter) == 1) { // double letter

            if(this.board[x][y].equals("DL")){
            counter += 2;
            }else if(this.board[x][y].equals("TL")){
                counter += 3;
            }else if(this.board[x][y].equals("NaN") || this.board[x][y
                counter += 1;
            }

        }else {
            if(this.board[x][y].equals("DL")){
            counter += 2 * letterScores.get(currentLetter);
            } else if(this.board[x][y].equals("TL")){
                counter += 3 * letterScores.get(currentLetter);
            } else if(this.board[x][y].equals("NaN") || this.board[x][y
                counter += letterScores.get(currentLetter);
```

```
            }
        }
        return counter;
    }

    int mulScore(int x, int y){

        if (this.board[x][y].equals("DW")){
            return 2;
        } else if(this.board[x][y].equals("TW")){
            return 3;
        } else {
            return 0;
        }
    }
```

Weiterhin werden im Konstruktor die “bags” für Player01 und Player02 gefüllt. Zum Holen eines Steins vom Board, wird “tile” auf leer gesetzt und tileBoard auf den char vom Spielfeld an der geklickten Position.

Scrabble01 Click-Event

Nach jedem Click wird eine Reihe von Anweisungen durchgeführt. Dazu wird den Spielenden zuerst die Möglichkeit gegeben einmalig das Level auszuwählen, mit einem boolean-Feld wird kontrolliert, dass es nur einmal und direkt am Anfang ausgewählt werden kann. Nach dem ersten Click, wird “isLevelChosen” auf true gesetzt und falls keine andere Einstellung gewählt wurde, die Schwierigkeit “normal” als default-Wert verwendet.

```
            if(!this.isLevelChosen){
                chooseLevel(x, y);
            }
```

Wird eine Einstellung gewählt, wird die Häufigkeit der Buchstaben in bag neu gesetzt, sowie der counter für die Hilfsfunktion auf den vorgegebenen Wert gesetzt.

```
    void chooseLevel(int x, int y){
```

```
        if(x >= 16 && x <= 18 && y == 5){
            this.level = Level.EASY;
            bag.level = Level.EASY;
            bag.level = Level.EASY;
        } else if(x >= 16 && x <= 18 && y == 6){
            this.level = Level.NORMAL;
            bag.level = Level.NORMAL;
            bag.level = Level.NORMAL;
        } else if(x >= 16 && x <= 18 && y == 7){
            this.level = Level.HARD;
            bag.level = Level.HARD;
            bag.level = Level.HARD;
        }
        bag.setFrequnecies();
        this.setHelpFrequency();
        Clerk.script(view, "scrabble" + ID + ".drawCounter1(" + this.c
        Clerk.script(view, "scrabble" + ID + ".drawCounter2(" + this.c
        this.isLevelChosen = true;
    }
```

Nun wird das Scrabble-Feld nach dem aktuellen Stand neu gezeichnet, um Fehlermeldungen nach dem letzten Legen wieder zu entfernen.

Für den Fall, dass auf den Button ‚change" gedrückt wird, werden die gegebenen Buchstabenplättchen (tilesTop, tilesBottom) ausgetauscht und random mit Buchstaben aus dem Säckchen des aktuellen Spielers befüllt. Die Bedingung, um die Steine tauschen zu können, ist dass in dem Zug noch kein Stein auf das Spielfeld gelegt wurde, da sonst beliebig getauscht und auf das Feld gelegt werden könnte.

```
        void refillTiles(int x, int y){ //tausche alle tiles aus, aber
            Random random = new Random();
            boolean refill = true;
            if(x >= 16 && x <= 18 && y == 13){
                if(this.currentPlayer > 0){
                    for(char tile : this.tilesTop){
                        if(tile == '0'){ //man darf nur vor dem Zug ei
                            refill = false;
                            break;
                        }
                    }
```

```
                if(refill){
                    for (int i = 0; i < this.tilesTop.length; i++)
                        player1.bag.add(this.tilesTop[i]); //befül
                        int rand = random.nextInt(player1.bag.size
                        this.tilesTop[i] = player1.bag.get(rand);
                        player1.bag.remove(rand);
                        }
                        drawTiles();
                    }
                } else if(this.currentPlayer < 0){
                    for(char tile : this.tilesBottom){
                        if(tile == '0'){
                            refill = false;
                            break;
                        }
                    }
                    if(refill){
                        for (int i = 0; i < this.tilesBottom.lengt
                        player2.bag.add(this.tilesBottom[i]);
                        int rand = random.nextInt(player2.bag.size
                        this.tilesBottom[i] = player2.bag.get(ranc
                        player2.bag.remove(rand);
                        }
                        drawTiles();
                    }
                }
            }
        }
```

Nun werden die Methoden getTile() und updateBoard() aufgerufen, welche zum Aufheben eines Buchstabenplättchens und zum Setzen von diesem ist. Zum Holen eines Steins von tilesTop/ tilesBottom, wird ‚tile" auf den genommenen char gesetzt, während tileBoard auf leer (’ ') gesetzt wird.

```
char getTile(int x, int y) {
    // Hole Stein für ersten Klick (von TopTiles)
    if (x >= 7 && x <= 13 && y == 2 && this.currentPlayer > 0 && h
        this.tile = this.tilesTop[x-7];  // Hole Stein auf den gek
        this.tileBoard = ' ';
        this.tilesTop[x-7] = '0'; // Entferne Stein von TopTiles
```

```
        Clerk.script(view, "scrabble" + ID + ".removeTile("+ x + '
    }
```

SZENARIO 1: WORTVALIDIERUNG, SZENARIO 2: PUNKTEZÄHLUNG

Im ersten Zug wird das erste Wort zusammengesetzt und in “firstWord” gespeichert. Es werden in fPos die ersten Clicks gespeichert, solange sie in einer Reihe oder Spalte sind. Falls noch mal auf die gleiche Position geklickt wird, entferne diese wieder. Wenn also auf eine Position geklickt wurde, die bereits in fPos gespeichert ist. Wenn außerhalb des Feldes geklickt wird, beispielsweise auf play, dann wird in den else-Fall gegangen und die Positionen werden sortiert, bevor das Wort aus den gespeicherten Positionen zusammengesetzt wird. Dann wird geprüft, ob das erste Wort validiert wird. Ist dies der Fall, wird es zu validatedWords hinzugefügt, sonst zu wrongWords.

```
void getFirstWord(int x, int y){
    if(this.isFirstWord && x >= 0 && x <= 14 && y >= 3 && y <= 17)

    int lastPos = 0;
    y -= 3;

    if(this.firstWord.positions.size() > 1){
        lastPos = this.firstWord.positions.size()-2;
        }
        if(!(this.firstWord.positions.size() > 0)) {
            this.firstWord.addPosition(x, y);
        } else {
            if(((x) == (this.firstWord.positions.get(lastPos).x) &
                if(this.firstWord.positions.contains(new Position(
                    this.firstWord.positions.remove(new Position(x
                } else if(!(this.firstWord.positions.contains(new
                    this.firstWord.addPosition(x, y);
                }
            } else {
                this.isFirstWord = false;
                StringBuilder fWord = new StringBuilder();
                Collections.sort(this.firstWord.positions);
```

```
                    for(Position pos: this.firstWord.positions){
                        x = pos.x;
                        y = pos.y;
                        fWord.append(this.updatedBoard[x][y]);
                    }
                    this.firstWord.word = fWord.toString();
                    this.firstWord.start = this.firstWord.positions.ge
                    this.firstWord.end = this.firstWord.positions.get

                    if(validateWord(this.firstWord.word)){
                        this.validatedWords.add(this.firstWord);
                    } else {
                        this.wrongWords.add(this.firstWord);
                        this.isFirstWord = true;
                    }
                }
            }
        }
    }
```

Wenn auf den Button “play” geklickt wird, wird in den endTurn()-Abschnitt gegangen. Die Inhalte aus dem currentBoard werden auf das updatedBoard übertragen. Wenn in validatedWords noch keine Einträge sind, wird anhand der Positionen des ersten Worts die mit dem firstWord zusammenhängenden Wörter vom Spielfeld extrahiert (mit getWords()). Wenn in validatedWords Einträge sind, dann wird dies anhand der Positionen in validatedWords getan. Mit allen Positionen vom Board (boardPositions), werden alle Wörter die auf dem Feld stehen, extrahiert.

```
            if(endTurn(x, y)){
                overrideUpdatedBoard(); //currentBoard wird hinzugefüg
                if(this.validatedWords.size() > 0) {
                    List<Position> validatedPositions = new ArrayList<
                    for (Word w : this.validatedWords) {
                        validatedPositions.addAll(w.positions); // All
                    }
                    this.wrongWords = getWrongWords(getWords(this.upda
                } else {
                    this.wrongWords = getWrongWords(getWords(this.upda
            }
```

Methode getWrongWords

In der Methode getWrongWords werden die Inhalte dieser beiden Listen verglichen, um die Wörter zu erhalten die nicht an den anderen Wörtern hängen. Diese sind nicht erlaubt, dementsprechend werden sie zu wrongWords hinzugefügt, sodass eine Fehlerausgabe stattfinden kann. Die anderen Wörter werden auf ihre Validierung überprüft. Wenn sie validiert werden zu validatedWords hinzugefügt, sonst zu wrongWords.

```
List<Word> getWrongWords(List<Word> allWords, List<Word> words){
    List<Word> allWords2 = new ArrayList<>(allWords);
    List<Word> words2 = new ArrayList<>(words);
    List<Word> vWords = new ArrayList<>();

    Collections.sort(allWords2);
    Collections.sort(words2);

    for (int i = 0; i < allWords2.size(); i++) {
        for (int j = 0; j < words2.size(); j++) {
            if(allWords2.get(i).word.equals(words2.get(j).word)){
                vWords.add(allWords2.get(i)); //füge das Wort in v
                allWords2.remove(i); // in allWords stehen nicht-z
            }
        }
    }
    if(validateWord(this.firstWord.word)){ //wenn firstWord richti
        validatedWords.add(this.firstWord);
        if(allWords2.contains(this.firstWord)){
            allWords2.remove(allWords2.indexOf(this.firstWord));
        }
    }
    for (Word word : words2) {
        if(!validateWord(word.word)){
            allWords2.add(word);
        }
    }
    this.wrongWords = allWords2;
    if(this.wrongWords.isEmpty()){ //validierte Wörter um in nächs
        for(Word w : vWords){
            this.validatedWords.add(w);
        }
```

```
        }
        return this.wrongWords;
    }
```

Methode getWords

Um zu vermeiden, dass Wörter mehrmals gespeichert werden, wird ein HashSet “uniqueWords” verwendet. Es werden alle an die Methode übergebenen Positionen durchlaufen.

Methoden extractWordHorizontal/ vertical

Auf jede einzelne Position werden die Methoden extractWordsHorizontal und extractWordsVertical aufgerufen, in denen von der gegebenen Position aus nach links bzw. nach oben gelaufen wird, soweit bis im Board eine “0” steht. Dann wird das Wort nach rechts, bzw. unten zusammengesetzt und das Wort mit seinen Positionen erstellt.

```
Word extractWordHorizontal(int x, int y, List<Position> posi, Stri
    StringBuilder w = new StringBuilder();
    List<Position> positions = new ArrayList<>();

    while (y > 0 && !(board1[x][y - 1].equals("0"))){
        y--;
    }

    Position start = new Position(x, y);

    //gehe nach rechts setze wort zusammen
    while (y < board1.length && !(board1[x][y].equals("0"))) {
        w.append(board1[x][y]);
        positions.add(new Position(x, y));
        y++;
    }
    Position end = new Position(x, y-1);
    Word word2 = new Word(w.toString(), start, end);
    word2.positions = positions;
    return word2;
}
```

Wenn die Länge des extrahierten Wortes größer als 1 ist, wird das Wort zu uniqueWords

hinzugefügt. Da einzelne Buchstaben nicht validiert werden sollen, müssen diese für die Fehlermeldung gespeichert werden. Wenn ein einzelner Buchstabe extrahiert wurde, wird überprüft, ob die herumliegenden Felder leer sind. Falls sie leer sind, wird der Buchstabe unique-Words hinzugefügt. Die Methode gibt eine Liste der extrahierten Wörter zurück.

Nachdem jetzt die wrongWords und validatedWords mithilfe von getWords() gespeichert wurden, wird überprüft, ob überhaupt falsche Wörter auf dem Spielfeld liegen (indem geschaut wird, ob wrongWords leer ist). Ist dies der Fall, kann der aktuelle Zug abgeschlossen werden. Die Plättchen werden aufgefüllt, der neue Score gezeichnet, das Board mit den gelegten Buchstaben aktualisiert, das currentBoard geleert und der turn() durchgeführt (durch *(-1) des currentPlayers). Player1 = 1, Player2 = -1.

Im anderen Fall (wrongWords ist nicht empty), wird eine Fehlermeldung auf das Board gezeichnet. Die Liste wrongWords wird geleert und die gelegten Buchstaben werden aus dem updatedBoard entfernt, indem die Buchstaben aus dem currentBoard vom updatedBoard gelöscht werden.

Methode validateWords

Hier findet die Überprüfung der Wörter statt. Es wird auf die Wörterbuch-API vom DWDS zugegriffen. Zwei inputs für die Methode isValidWord werden bestimmt, einmal das übergebene Wort mit großem Anfangsbuchstaben und einmal das Wort klein geschrieben. Wenn isValidWord bei einem der beiden Eingaben true ausgibt, so wird das Wort validiert.

```
boolean validateWord(String input) {
    // Großer Anfangsbuchstabe, kleiner Anfangsbuchstabe. Wenn eir
    if(input.length() < 2){
        return false;
    } else {

    String input1 = input.substring(0, 1).toUpperCase() + input.su
    String input2 = input.toLowerCase();

    // Funktion für die Validierung des Wortes
    return isValidWord(input1) || isValidWord(input2);
    }
}
```

Methode isValidWord

Es wird ein GET-Request an die DWDS-API gesendet, wobei eine URL verwendet werden kann an die hinten der input angefügt wird. Bei richtiger Eingabe kommt die Antwort im JSON-Format. Beispiel: [{“input”:“Puma”,“url”:“https://www.dwds.de/wb/Puma",“lemma”:“Puma”,“wortart”:"Substantiv”}] Deshalb wird überprüft ob die Antwort ein lemma enthält und der boolean zurückgegeben.

```
private boolean isValidWord(String word) {
    try {
        URI uri = URI.create("https://www.dwds.de/api/wb/snippet/?
        URL url = uri.toURL();
        HttpURLConnection connection = (HttpURLConnection) url.ope
        connection.setRequestMethod("GET");

        BufferedReader in = new BufferedReader(new InputStreamReac
        StringBuilder response = new StringBuilder();
        String inputLine;

        while ((inputLine = in.readLine()) != null) {
            response.append(inputLine);
        }
        in.close();

        // JSON als String durchsuchen
        String jsonResponse = response.toString();
        return jsonResponse.contains("\"lemma\":");
    } catch (Exception e) {
        e.printStackTrace();
        return false;
    }
}
```

Beispiel-Anzeige für Szenario 1 und 2 (Wortvalidierung und Punktezählung)

Spielfeld 1: Das Wort ‚Baum" wurde validiert und die Score-Anzeige auf 8 erhöht.

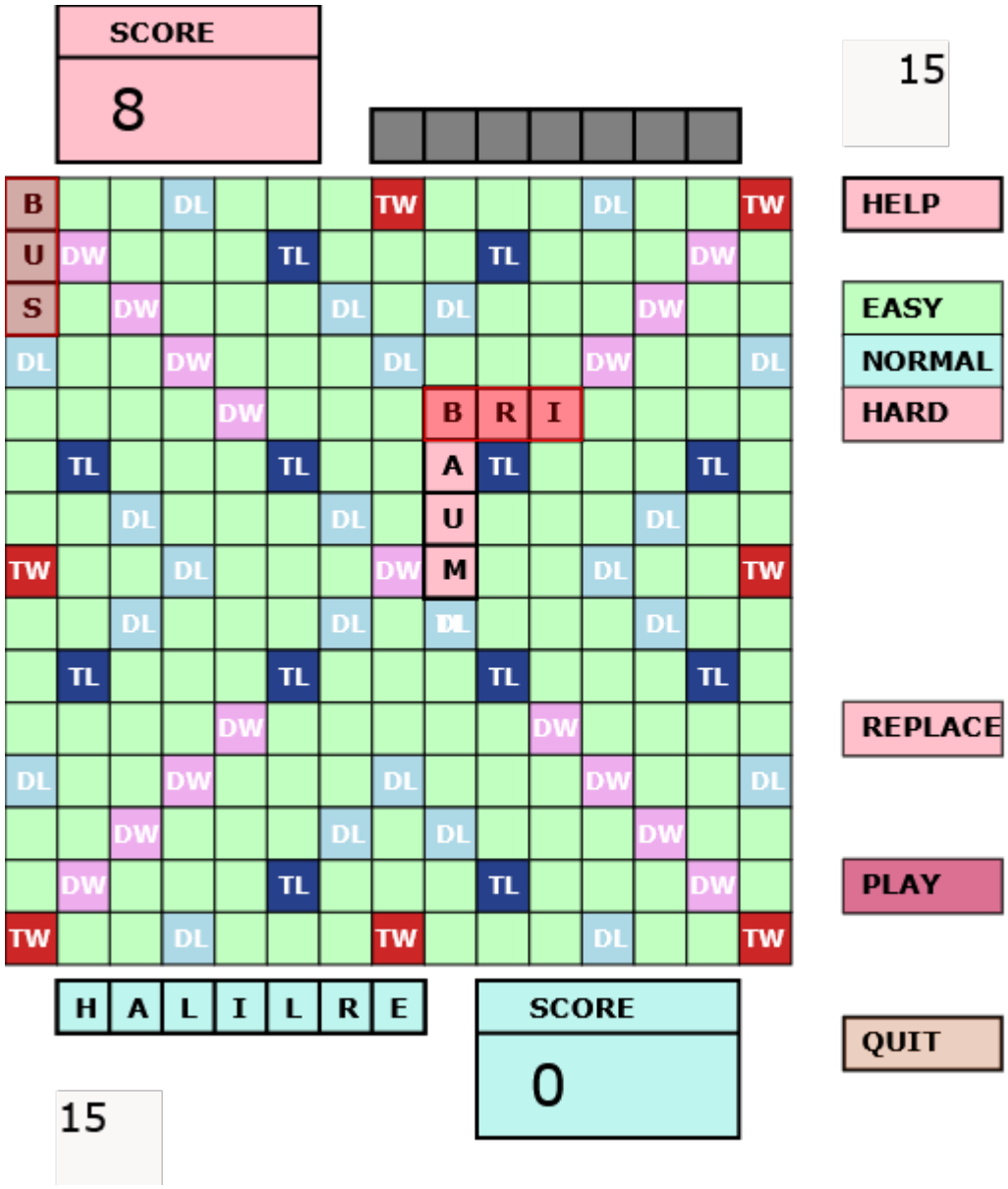
```
Rechnung:

B = 3
```

```
A = 1
U = 1 = DL = 2
M = 2
= 3 + 1 + 2 + 2 = 8
```

- scrabble.validateWord(“BAUM”) $89 ==> true

Nun wurden das Wort “BRI” hinzugefügt, an welchem die Fehlermeldung zu sehen ist. Auch das Wort “Bus” wird nicht validiert, da es nicht am ersten Wort “BAUM” hängt.



SZENARIO 3: HILFSFUNKTION

Die Hilfsdunktion funktioniert mit einer Wortliste, welche Anfangs in ein HashSet

geladen wird. Nun werden aus den aktuellen Buchstaben des Spielers alle möglichen Kombinationen erstellt. Dann werden aus allen Kombinationen alle möglichen Permutationen erstellt.

Es wird nun in der eigentlichen Hilfsfunktionen für jede dieser Permutationen geprüft, ob sie in dem HashSet der Wortliste enthalten sind. Das hashSet wurde gewählt, da dort die Laufzeit für contains in O(1) liegt. Da die gewählte Wortliste möglicherweise Wörter enthält, die von der DWDS-API nicht validiert werden, wird dies in der doHelpFunc() nocheinmal überprüft. Durch die direkte Validierung aller Permutationen durch die API, würde die Laufzeit sehr lang werden, deshalb die Lösung durch die Wortliste. Nun wird aus den validierten Wörtern das Wort mit dem höchsten Score ausgewählt und die Ausgabe auf das Spielfeld gezeichnet.

Die Funktion generateCombinations funktioniert rekursiv. Es werden zwei Strings übergeben, einmal der String aus dem die Kombinationen generiert werden sollen (letterString) und einmal ein leerer String. Die Abbruchbedingung der Rekursion ist, dass letterString leer ist. Dann wird empty zu der Liste von Kombinationen hinzugefügt und diese Liste zurückgegeben.

Nun wird in den rekursiven Fall gegangen:

Die Funktion ruft sich selbst auf mit dem Teilstring von letterString, der am zweiten Buchstaben beginnt. Für Baum also “aum” und mit dem ersten Buchstaben also “B”. Weitere Aufrufe:

Erster Rekursionszweig (Buchstabe wird hinzugefügt):

```
"aum", "B"
    "um", "Ba"
        "m", "Bau"
            "", "Baum"
"um", "Ba"
    "m", "Ba"
        "", "Bam"
```

Zweiter Rekursionszweig (Buchstabe wird ignoriert):

```
"aum", ""
    "um", ""
        "m", ""
```

```
            "", ""
    "m", ""
        "", "m"
```

[“Baum”, “Bau”, “Bam”, “Ba”, “Bum”, “Bu”, “Bm”, “B”, “aum”, “au”, “am”, “a”, “um”, “u”, “m”, “”]

```
List<String> generateCombinations(String letterString, String empt
    List<String> combintions = new ArrayList<>();

        if (letterString.isEmpty()) {
            combintions.add(empty);
            return combintions;
        }
        // 1. Den ersten Buchstaben hinzufügen
        combintions.addAll(generateCombinations(letterString.subst
        // 2. Den ersten Buchstaben nicht hinzufügen
        combintions.addAll(generateCombinations(letterString.subst
        return combintions;
    }
```

Aus diesen Kombinationen werden nun alle Permutationen erstellt. In der Methode Permutation() werden alle Kombinationen des Wortes in einer ArrayList gespeichert, woraufhin aus jeder Kombination deren Permutationen erstellt wird durch den Aufruf der Funktion permute(). permute wird aufgerufen mit dem char-Array, also der Kombination aus topTiles/bottom, einem ganzzhligen Startwert und der result-Liste, die in Permutation zurückgegeben werden soll. Beim ersten Aufruf wird start mit 0 aufgerufen. Die Abbruchbedingung ist, dass start gleich der Position des letzten Buchstabens aus dem char-Array ist. Nun laufen wir von start bis zum ende des char-Arrays in einer for-Schleife und tauschen das Element an i mit start. Es folgt der rekursive Aufruf, wobei start um 1 erhöht wird. Danach werden die Buchstaben zurückgetauscht.

Das bedeutet durch das inkrementieren von start, wird jeder Buchstabe einmal mit i getauscht, dann wird i erhöht usw.

Aufruf mit [BAUM] i = 0: [ABUM] [UABM] [MAUB] …

```
List<String> permutation(char[] letters) {
    List<String> results = new ArrayList<>();
    String lettersString = charToString(letters);
```

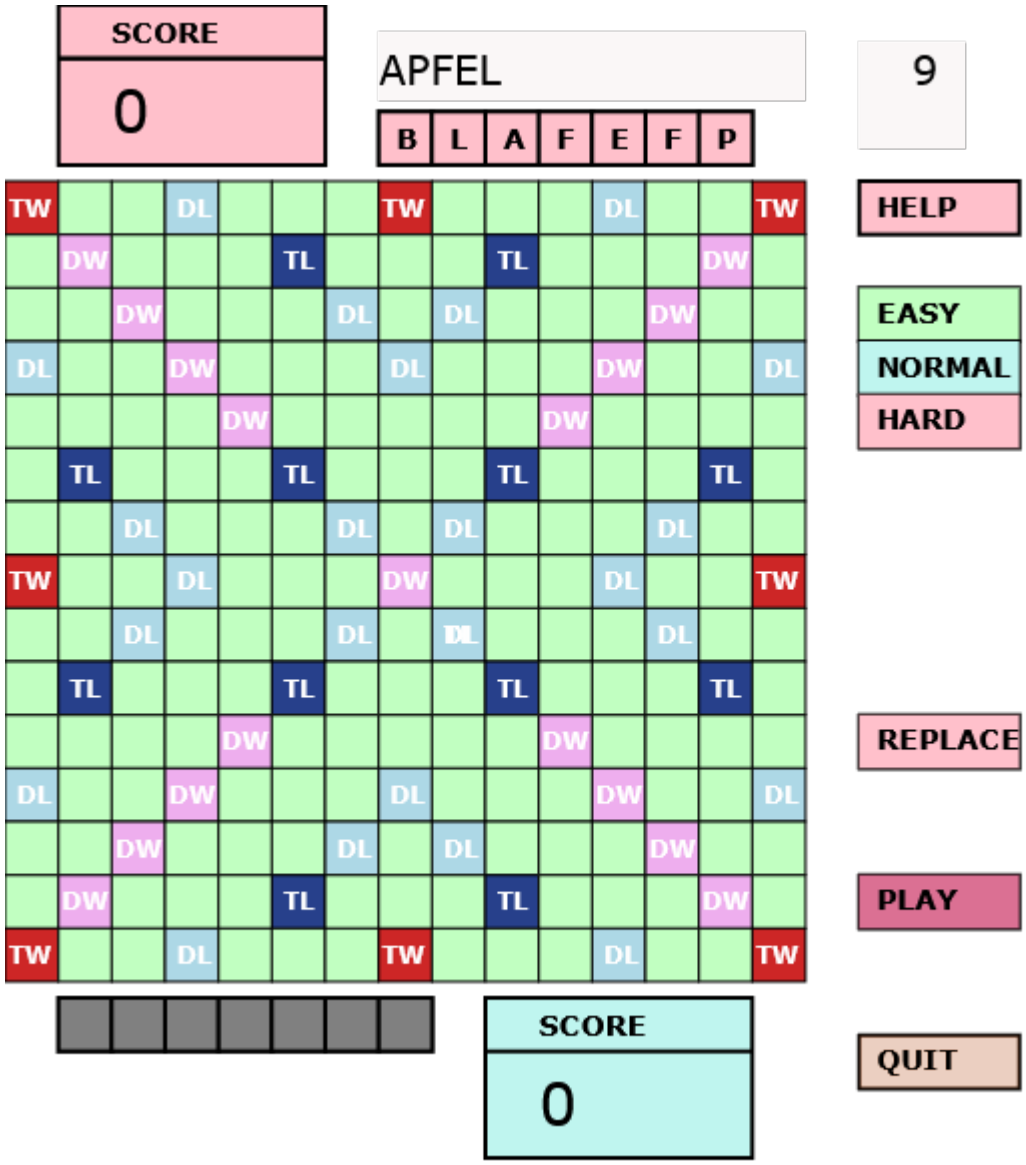
```
        List<String> combinations = generateCombinations(lettersStrinç

        for (String combination : combinations) {
            char[] combinationArray = combination.toCharArray();
            permute(combinationArray, 0, results);
        }
    return results;
    }

    void permute(char[] chars, int start, List<String> results) {

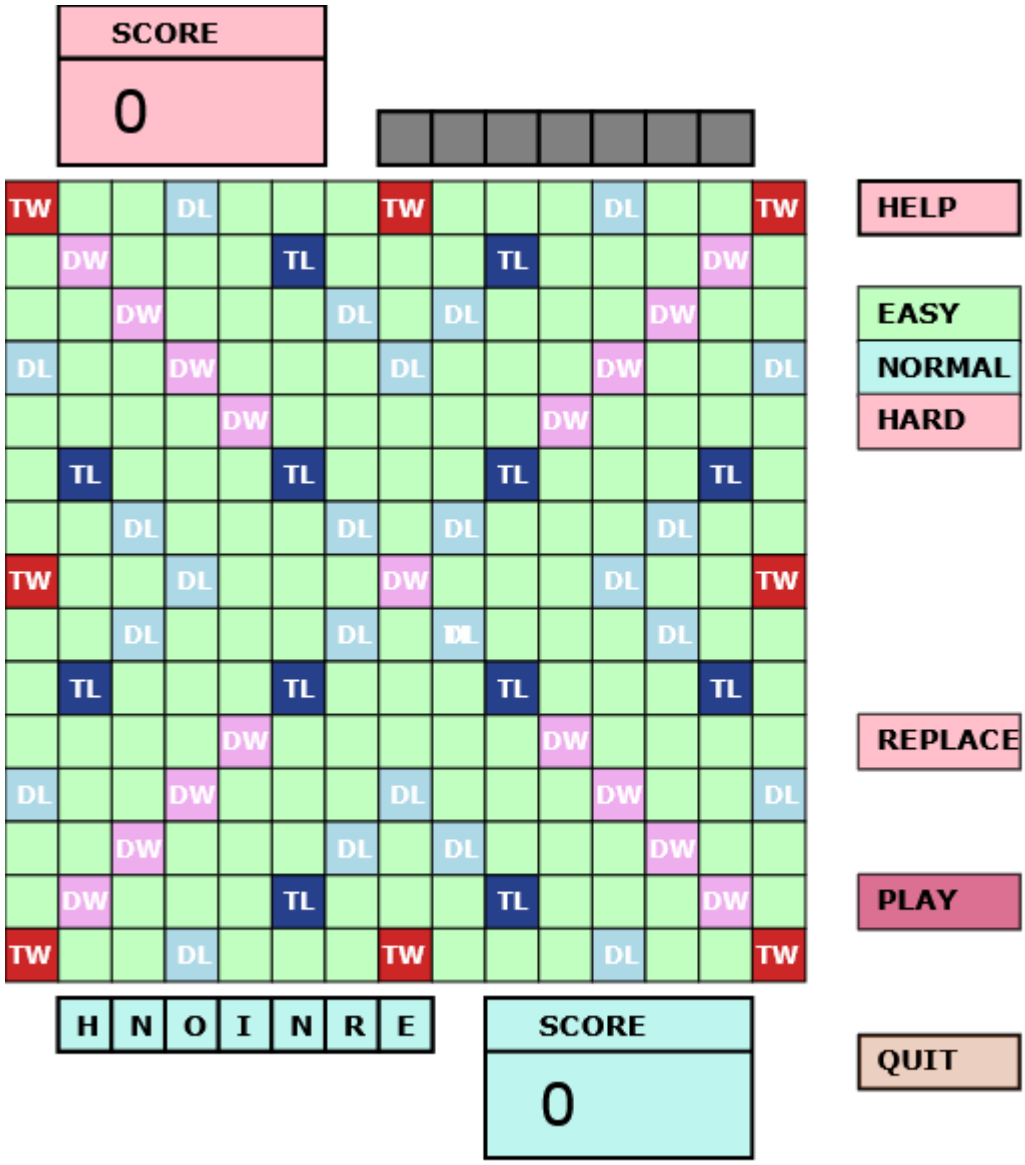
        if (start == chars.length - 1) {
            String c = charToString(chars);
            results.add(c); // Aktuelle Permutation speichern
            return;
        }

        for (int i = start; i < chars.length; i++) {
            swap(chars, start, i);
            permute(chars, start + 1, results);
            swap(chars, start, i);
        }
    }
```



SZENARIO 4: UMDREHEN DER BUCHSTABENPLÄTTCHEN

Es sind nur die Buchstabenplättchen des Spielers offengelegt, der aktuell an der Reihe ist, damit für die Spielenden klar zu erkennen ist, wer am Zug ist. Im Beispiel ist Spieler2 dran, die Plättchen von Spieler1 sind verdeckt.



SZENARIO 5: ENDE DES SPIELS

Es wird geprüft, ob das Spiel zuende ist. Die Bedingungen für das Ende des Spiels sind: - Es wurde auf Aufgeben (Quit) gedrückt. - Der Beutel eines Spielers sowie tilesTop für Spieler1 und tilesBottom für Spieler2 sind leer.

Wenn eine der Bedingungen erfüllt ist, wird das Spiel beendet, indem eine Nachricht ausgegeben wird, welcher Spieler gewonnen hat oder dass das Spiel unentschieden geendet hat.

```
boolean isGameOver(int x, int y){
    if((this.player1.bag.size() == 0 && areTilesTopEmpty()) || (th
```

```
            gameIsntOver = false;
            return true;
        }
        return false;
    }
```

